Actividad 3: Automatización e Integridad con Triggers

Cristóbal Suárez Abad

2º ASIR

Administración de Sistemas Gestores de Bases de Datos

Objetivo: Implementar mecanismos automáticos para mantener la integridad de la información y reaccionar a eventos clave en el sistema de alquiler de videos.

Utilizando dvdrental, realiza lo siguiente. Comprueba que funciona como se espera.

Ejercicio 1. Evita que se alquile una película si el cliente tiene multas pendientes.

Ejercicio 2. Registrar en una tabla de log cada vez que se cambie el precio de alquiler (`rental_rate`) de una película.

Ejercicio 3: No debemos permitir que se inserte un pago (`payment`) con una cantidad (`amount`) negativa, ya que afectaría la consistencia financiera.

Ejercicio 4: Para mantener la integridad histórica, queremos guardar un registro cada vez que la dirección de un cliente (address en la tabla customer) sea actualizada.

Crea la función fn_log_cambio_direccion() y el trigger correspondiente que, AFTER UPDATE, inserte en customer_log_cambios el ID del cliente y los valores de la dirección antigua (OLD.address_id) y la nueva (NEW.address_id) junto con la fecha del cambio.

Entrar en PostgreSQL:

```
sudo -i -u postgres psql
```

Crea scripts desde fuera de PostgreSQL y luego añádelos a la base de datos “dvdrental” con este comando:

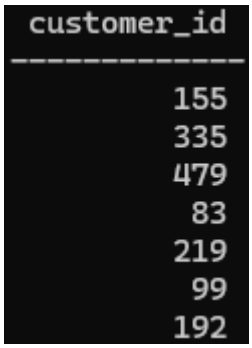
```
psql -U postgres -h localhost -d dvdrental -f “NOMBRE_SCRIPT.sql”
```

Se indica usuario (-U), el localhost indica que es el a misma máquina y el “-d” la base de datos.

Ejercicio 1. Evita que se alquile una película si el cliente tiene multas pendientes.

Saber ID de los clientes con alquileres pendientes:

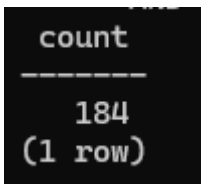
```
SELECT customer_id
   FROM rental
   WHERE return_date IS NULL
     AND rental_date < CURRENT_DATE;
```



```
 customer_id
-------------
         155
         335
         479
          83
         219
          99
         192
```

Saber el total de clientes con alquileres pendientes:

```
SELECT COUNT(*)
   FROM rental
   WHERE
      return_date IS NULL
     AND rental_date < CURRENT_DATE;
```



```
 count
-------
   184
(1 row)
```

```
-- 1. Función de validación
CREATE OR REPLACE FUNCTION fn_revisar_multas_cliente()
RETURNS TRIGGER AS $$
-- https://www.postgresql.org/docs/current/pltcl-trigger.html
-- Indica que esta función no devuelve un número o texto común,
-- sino que está diseñada específicamente para ser disparada por un Trigger.
-- El $$ es un delimitador de inicio de bloque de código.
DECLARE
    v_retrasos INT;
BEGIN
    -- Contamos alquileres que no han sido devueltos y han pasado la fecha límite
    -- Que pertenezca al customer nuevo, que no se haya devuelto (NULL) y que la
fecha de alquiler sea menor que la actual.
    SELECT COUNT(*) INTO v_retrasos
    FROM rental
    WHERE customer_id = NEW.customer_id
      AND return_date IS NULL
      AND rental_date < CURRENT_DATE;

    IF v_retrasos > 0 THEN
        RAISE EXCEPTION 'El cliente con ID % tiene alquileres pendientes fuera de plazo. No
puede alquilar.', NEW.customer_id;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- 2. Trigger
CREATE TRIGGER tr_check_multas_antes_alquiler
-- Que se active antes del nuevo registro.
BEFORE INSERT ON rental
-- Por si se hacen inserciones masivas.
FOR EACH ROW
-- Conecta el trigger con la función anterior
EXECUTE FUNCTION fn_revisar_multas_cliente();
```

Vamos a probarlo con uno de los usuarios que salían antes, el 155. Insertamos fecha (Ahora), ID de inventario, la ID del cliente y alguien del staff.

```
INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)

VALUES (NOW(), 1, 155, 1);
```

```
dvdrental=# INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)
VALUES (NOW(), 1, 155, 1);
ERROR:  El cliente con ID 155 tiene alquileres pendientes fuera de plazo. No puede alquilar.
CONTEXTO:  PL/pgSQL function fn_revisar_multas_cliente() line 13 at RAISE
```

Ejercicio 2. Registrar en una tabla de log cada vez que se cambie el precio de alquiler (`rental_rate`) de una película.

Usamos “**\dt**” para ver las tablas de la base de datos.

```
dvdrental-# \dt
                List of relations
 Schema |          Name           | Type  |  Owner
--------+-------------------------+-------+----------
 public | actor                   | table | postgres
 public | address                 | table | postgres
 public | category                | table | postgres
 public | city                    | table | postgres
 public | country                 | table | postgres
 public | customer                | table | postgres
 public | customer_log_cambios    | table | postgres
 public | film                    | table | postgres
 public | film_actor              | table | postgres
 public | film_category           | table | postgres
 public | inventory               | table | postgres
 public | language                | table | postgres
 public | log_precios_peliculas   | table | postgres
 public | payment                 | table | postgres
 public | rental                  | table | postgres
 public | staff                   | table | postgres
 public | store                   | table | postgres
(17 rows)
```

Usamos “**\d film**” para ver las columnas de la tabla.

```
dvdrental-# \d film
                                        Table "public.film"
     Column      |            Type             | Collation | Nullable |                  Default
-----------------+-----------------------------+-----------+----------+-------------------------------------------
 film_id         | integer                     |           | not null | nextval('film_film_id_seq'::regclass)
 title           | character varying(255)      |           | not null |
 description     | text                        |           |          |
 release_year    | year                        |           |          |
 language_id     | smallint                    |           | not null |
 rental_duration | smallint                    |           | not null | 3
 rental_rate     | numeric(4,2)                |           | not null | 4.99
 length          | smallint                    |           |          |
 replacement_cost | numeric(5,2)               |           | not null | 19.99
 rating          | mpaa_rating                 |           |          | 'G'::mpaa_rating
 last_update     | timestamp without time zone |           | not null | now()
 special_features | text[]                     |           |          |
 fulltext        | tsvector                    |           | not null |
```

```
-- 1. Creamos una tabla para el log donde guardamos datos básicos
-- de la película y los precios (antiguo y nuevo)
CREATE TABLE IF NOT EXISTS log_precios_peliculas (
    log_id SERIAL PRIMARY KEY,
    film_id INT,
    titulo VARCHAR(255),
    precio_anterior NUMERIC(4,2),
    precio_nuevo NUMERIC(4,2),
    fecha_cambio TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Función
-- La función se activa si el precio antiguo es diferente al nuevo.
-- Si es diferente insertará los datos en la tabla de log.
CREATE OR REPLACE FUNCTION fn_log_cambio_precio()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.rental_rate <> NEW.rental_rate THEN
        INSERT INTO log_precios_peliculas (film_id, titulo, precio_anterior, precio_nuevo)
        VALUES (OLD.film_id, OLD.title, OLD.rental_rate, NEW.rental_rate);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- 3. Trigger
-- De igual manera que en el anterior.
-- pero esta vez es AFTER UPDATE, para que guarde el cambio
-- Creamos trigger y que active la función anterior.
-- También está hecho para que funcione con actualizaciones masivas.
CREATE TRIGGER tr_audit_precio_pelicula
AFTER UPDATE OF rental_rate ON film
FOR EACH ROW
EXECUTE FUNCTION fn_log_cambio_precio();
```

Probamos con:

```
UPDATE film SET rental_rate = 4.99 WHERE film_id = 1;
SELECT * FROM log_precios_peliculas;
```

```
dvdrental=# UPDATE film SET rental_rate = 4.99 WHERE film_id = 1;
SELECT * FROM log_precios_peliculas;
UPDATE 1
 log_id | film_id |      titulo      | precio_anterior | precio_nuevo |        fecha_cambio
--------+---------+------------------+-----------------+--------------+----------------------------
      1 |       1 | Academy Dinosaur |            0.99 |         4.99 | 2026-05-12 10:20:35.365943
(1 row)
```

Vemos que el precio se ha cambiado en la tabla original.

```
SELECT film_id, title, rental_rate FROM film where film_id = 1;
```

```
dvdrental=# SELECT film_id, title, rental_rate FROM film where film_id = 1;
 film_id |      title       | rental_rate
---------+------------------+-------------
       1 | Academy Dinosaur |        4.99
(1 row)
```

Si volvemos a cambiar el precio del alquiler, también se registrará.

```
dvdrental=# SELECT * FROM log_precios_peliculas;
 log_id | film_id |      titulo      | precio_anterior | precio_nuevo |        fecha_cambio
--------+---------+------------------+-----------------+--------------+----------------------------
      1 |       1 | Academy Dinosaur |            0.99 |         4.99 | 2026-05-12 10:20:35.365943
      2 |       1 | Academy Dinosaur |            4.99 |         5.00 | 2026-05-13 17:22:55.966753
      3 |       1 | Academy Dinosaur |            5.00 |         5.20 | 2026-05-13 17:23:07.530051
      4 |       1 | Academy Dinosaur |            5.20 |         6.20 | 2026-05-13 17:23:17.200331
(4 rows)
```

Ejercicio 3: No debemos permitir que se inserte un pago (`payment`) con una cantidad (`amount`) negativa, ya que afectaría la consistencia financiera.

```
\d payment
```

```
dvdrental=# \d payment
                                        Table "public.payment"
    Column    |            Type             | Collation | Nullable |
--------------+-----------------------------+-----------+----------+-
 payment_id   | integer                     |           | not null |
 customer_id  | smallint                    |           | not null |
 staff_id     | smallint                    |           | not null |
 rental_id    | integer                     |           | not null |
 amount       | numeric(5,2)                |           | not null |
 payment_date | timestamp without time zone |           | not null |
Indexes:
```

```
-- 1. Función
CREATE OR REPLACE FUNCTION fn_validar_pago_positivo()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.amount < 0 THEN
        RAISE EXCEPTION 'Error: No se permiten pagos con importes negativos (%).',
NEW.amount;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- 2. Trigger
CREATE TRIGGER tr_check_pago_positivo
BEFORE INSERT OR UPDATE ON payment
FOR EACH ROW
EXECUTE FUNCTION fn_validar_pago_positivo();
```

```
INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date)
VALUES (1, 1, 1, -5.00, NOW());
```

```
dvdrental=# INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date)
VALUES (1, 1, 1, -5.00, NOW());
ERROR:  Error: No se permiten pagos con importes negativos(-5.00).
CONTEXTO:  PL/pgSQL function fn_validar_pago_positivo() line 4 at RAISE
dvdrental=#
```

Ejercicio 4: Para mantener la integridad histórica, queremos guardar un registro cada vez que la dirección de un cliente (address en la tabla customer) sea actualizada.

Crea la función fn_log_cambio_direccion() y el trigger correspondiente que, AFTER UPDATE, inserte en customer_log_cambios el ID del cliente y los valores de la dirección antigua (OLD.address_id) y la nueva (NEW.address_id) junto con la fecha del cambio.

```
\d customer
```

```
dvdrental=# \d customer
                                        Table "public.customer"
   Column    |            Type             | Collation | Nullable |
-------------+-----------------------------+-----------+----------+-
 customer_id | integer                     |           | not null |
 store_id    | smallint                    |           | not null |
 first_name  | character varying(45)       |           | not null |
 last_name   | character varying(45)       |           | not null |
 email       | character varying(50)       |           |          |
 address_id  | smallint                    |           | not null |
 activebool  | boolean                     |           | not null |
 create_date | date                        |           | not null |
 last_update | timestamp without time zone |           |          |
 active      | integer                     |           |          |
Indexes:
```

```
-- 1. Tabla de Log de direcciones
CREATE TABLE IF NOT EXISTS customer_log_cambios (
    log_id SERIAL PRIMARY KEY,
    customer_id INT,
    old_address_id INT,
    new_address_id INT,
    fecha_cambio TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Función
CREATE OR REPLACE FUNCTION fn_log_cambio_direccion()
RETURNS TRIGGER AS $$
BEGIN
    -- Solo insertamos si el address_id realmente ha cambiado
    IF OLD.address_id <> NEW.address_id THEN
        INSERT INTO customer_log_cambios (customer_id, old_address_id, new_address_id)
        VALUES (OLD.customer_id, OLD.address_id, NEW.address_id);
    END IF;
    RETURN NEW;
--
https://www.reddit.com/r/PostgreSQL/comments/ifp2rn/why_do_trigger_functions_return_new/
-- Si la función llega al RETURN NEW, significa que pasó todos los filtros e IF previos.
-- Es decir "Todo está correcto, procede a guardar estos datos".
END;
$$ LANGUAGE plpgsql;

-- 3. Trigger
CREATE TRIGGER tr_log_direccion_cliente
AFTER UPDATE OF address_id ON customer
FOR EACH ROW
EXECUTE FUNCTION fn_log_cambio_direccion();
```

Probamos a actualizar una dirección:

```
UPDATE customer SET address_id = 5 WHERE customer_id = 1;
SELECT * FROM customer_log_cambios;
```

```
dvdrental=# UPDATE customer SET address_id = 5 WHERE customer_id = 1;
SELECT * FROM customer_log_cambios;
UPDATE 1
 log_id | customer_id | old_address_id | new_address_id |        fecha_cambio
--------+-------------+----------------+----------------+----------------------------
      3 |           1 |              6 |              5 | 2026-05-13 17:46:00.513276
(1 row)
```

select customer_id, first_name, last_name, address_id from customer where customer_id = 1;

```
dvdrental=# select customer_id, first_name, last_name, address_id from customer where customer_id = 1;
 customer_id | first_name | last_name | address_id
-------------+------------+-----------+------------
           1 | Mary       | Smith     |          5
(1 row)
```